

How Our Frontend And Backend Work Together

A plain guide for the Raiquid team

Written for the whole team to read before we connect the frontend to the live backend.

1. Why this guide exists

Our backend, raiquid api, is now live and fully tested. It handles every part of the business, the invoices, the buyers, the investors, the admin side, and it sends real emails and stores real files.

The frontend now needs to talk to it. This guide explains how that connection works, in plain words, so everyone on the team understands it the same way. It is not just for one person to read alone. Read it, and if something is not clear, ask before we start building on top of it.

There are four things we need to agree on and set up together. Each one is covered below, one at a time, with the exact steps to follow.

2. The big picture, two separate things happening

Before we touch any code, everyone needs to understand that there are two completely separate flows happening between our app and Clerk, the service that handles login and accounts. People often mix these two up, so we are going to be very clear about the difference from the start.

2.1 The everyday flow, this happens every time someone uses the app

Step 1. A person opens the app and signs in through Clerk.

Step 2. Clerk gives the browser something called a token. Think of a token as a small pass that proves who this person is, without them having to log in again for every single click.

Step 3. Any time the app needs to ask our backend for something, like a list of invoices, it attaches that pass to the request.

Step 4. Our backend checks that the pass is real and not expired before it does anything else.

Step 5. If the pass is good, the backend reads or writes data in the database and sends an answer back to the app.

This flow repeats over and over, every single time a person uses the app. It is the normal, everyday path.

2.2 The one time flow, this happens once, right when a person signs up

Step 1. A new person creates an account through Clerk.

Step 2. The moment that account is created, Clerk sends a message straight to our backend on its own, without the frontend being involved at all. This message is called a webhook.

Step 3. Our backend receives that message and creates a record for this person in our own database.

Step 4. That record is what tells our backend who this person is and what role they have, business, buyer, investor, or admin.

Step 5. This only happens once per person, at the moment they sign up. It has nothing to do with the browser or the frontend directly. It is backend talking to backend.

Here is the important part to remember and explain to the team. A person can have a valid pass from the first flow, but not yet have a record from the second flow. This can happen for a short moment right after someone signs up. When that happens, our backend will reply that the user is not set up yet. The correct answer on the frontend side is simply to wait a moment and try again. It is not a bug, it is expected.

3. Keeping our frontend types matched to the backend

The problem this solves

If someone on the frontend writes out the shape of a piece of data by hand, guessing what the backend sends back, that guess can quietly become wrong the moment the

backend changes something. Nothing will warn anyone. The app will simply break, sometimes without anyone noticing until a user reports it.

The fix

We do not guess. We generate the types straight from the backend itself. The backend already has a page that describes every piece of data it sends and receives, in exact detail. We point one small tool at that page, and it writes out a file with the exact shape of everything, automatically, in seconds.

Steps to follow

Step 1. Open a terminal inside the frontend project.

Step 2. Run the command below. It reads the backend description and writes out a file of matching types.

```
npx openapi-typescript \  
  https://raiquid-api-production.up.railway.app/docs-json \  
  -o src/types/api-generated.ts
```

Step 3. This creates one file containing the exact shape of every piece of data the backend uses.

Step 4. Never edit this file by hand. Put a short note at the very top of it saying so, so nobody forgets later.

Step 5. Any time the backend changes, for example a new field is added, run the same command again. The file updates itself to match.

Suggestion for the team. Add this command as a short script inside our package.json file, so everyone runs the same simple command instead of typing the long one by hand each time.

4. The services folder

The problem this solves

Without one shared place for talking to the backend, every screen that needs data would end up writing its own version of the same code, over and over. If we ever need to change how we attach the login pass, or how we handle an error, we would have to hunt for it everywhere in the app instead of fixing it in one place.

The fix

We keep one folder, with one file for each part of the backend. This also has a nice side effect. The folder ends up matching the backend folder almost exactly, so anyone on the team can guess where a piece of logic lives without needing to be told.

File	What it is for
src/services/client.ts	The one shared piece of code. It knows how to attach the login pass and handle errors. Every other file below builds on top of this one.
src/services/business.ts	Everything for the business side, submitting invoices, wallet, settings.
src/services/buyer.ts	Everything for the buyer side, reviewing and paying invoices.
src/services/investor.ts	Everything for the investor side, the marketplace, funding, wallet.
src/services/admin.ts	Everything for the admin side, the dashboards and the whitelisting review.
src/services/notifications.ts	Reading and marking notifications.

A short example of what a function inside one of these files looks like. Notice it does not deal with the login pass or the web address at all. It only asks for the data it needs, and it is fully typed using the file we generated in the section above.

```
import { apiClient } from './client';
import type { components } from '@types/api-generated';

type Invoice = components['schemas']['Invoice'];

export function createInvoice(
  data: CreateInvoiceInput
): Promise<Invoice> {
  return apiClient.post('/business/invoices', data);
}
```

Any screen in the app that needs to create an invoice simply calls createInvoice with the right data. It does not need to know anything about tokens, web addresses, or how errors are handled. That is all taken care of once, inside client.ts.

5. What goes in the frontend environment file

The environment file is where we keep small settings and keys that the app needs to run. It should only ever contain two things at this stage.

Name	Value	Why we need it
NEXT_PUBLIC_API_BASE_URL	Our backend web address	So the app knows where to send every request.
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY	The public key Clerk gave us	So Clerk can start the sign in process inside the browser.

An important rule to explain to the whole team

Anything in our environment file that starts with the word NEXT_PUBLIC is visible to anyone using the app, because it runs directly inside their own browser. Anything without that word in front stays hidden and only lives on our own server.

This is exactly why our secret Clerk key, along with our file storage key and our email key, must never be placed anywhere in the frontend project. They only ever live on the backend, where they are already set up and working.

Where a person's role gets decided

When someone signs up, we need to tell Clerk what role they have, business, buyer, investor, or admin. The simple way to do this is right inside the sign up code in the browser, at the moment of sign up.

```
await signUp.create({
  emailAddress,
  password,
  unsafeMetadata: { role: 'business' },
});
```

This small piece carries the role automatically through to the message our backend receives when the account is created. It needs no extra secret key on the frontend at all, which keeps our environment file simple and safe.

6. How to walk the team through this

When explaining this to the rest of the team, these are the four points to make, in this order.

Step 1. There are two separate flows between us and Clerk. One happens on every single request, the other happens once, at sign up. If a user gets a not set up yet reply, that is expected right after sign up, not a bug.

Step 2. We never write out the shape of backend data by hand. We generate it with one command, and we run that command again any time the backend changes.

Step 3. All talking to the backend goes through one services folder, never spread across random screens. One file per backend area, plus one shared file that handles the login pass.

Step 4. The environment file only ever holds two public values, the backend address and the public Clerk key. Every secret key stays on the backend, never on the frontend.

7. Quick checklist before we start

- [] Generate the types file from the live backend using the command in section 3.
- [] Set up the services folder with one file per backend area, as listed in section 4.
- [] Add the two environment variables listed in section 5 to the frontend project.
- [] Make sure the sign up code sets the correct role for every new account.
- [] Test one full sign up. Confirm a token comes back, and confirm a record appears on the backend a moment later.

Once every box above is checked, the frontend and backend are properly connected, and the rest of the team can start building screens on top of real, live data.